

Aim: To implement clustering algorithms for unsupervised classification.

1. How K-Means and Hierarchical Clustering Work ?

K-Means Clustering:

K-means is a partitioning method aiming to divide a dataset into K distinct, non-overlapping clusters. The algorithm works iteratively as follows:

1. **Initialization:** Randomly choose K cluster centroids.
2. **Assignment:** Assign each data point to the nearest centroid based on distance (typically Euclidean distance).
3. **Update:** Recalculate the centroids of each cluster by taking the mean of all data points assigned to that cluster.
4. **Repeat steps 2 and 3:** Iterate until the centroids stabilize or a maximum number of iterations is reached.

The goal is to minimize the within-cluster sum of squares, i.e., the sum of squared distances between each data point and its assigned centroid.

Hierarchical Clustering:

Hierarchical clustering builds a hierarchy of clusters by iteratively merging or splitting them. It can be agglomerative (bottom-up) or divisive (top-down).

• Agglomerative Hierarchical Clustering:

1. **Initialization:** Each data point is treated as a single cluster.
2. **Iteration:** At each step, the two closest clusters are merged based on a distance metric (e.g., Euclidean, Manhattan).
3. **Repeat:** This process continues until all data points belong to a single cluster, forming a dendrogram (tree-like structure) illustrating the hierarchy of clusters.

• Divisive Hierarchical Clustering:

1. **Initialization:** All data points belong to a single cluster.
2. **Iteration:** At each step, the cluster is divided into two sub-clusters based on a chosen criterion.
3. **Repeat:** This process continues recursively until each data point forms its own cluster.

2. Elbow Method and Silhouette Score

Elbow Method:

The elbow method is a heuristic used to determine the optimal number of clusters (K) in Kmeans clustering. It plots the within-cluster sum of squares (WCSS) against the number of clusters.

- WCSS represents the total variance within each cluster, with lower values indicating better clustering.
-
- The plot typically shows a decreasing trend as K increases.
The "elbow" point is where the rate of decrease sharply changes, indicating the point of diminishing returns in terms of improving clustering performance by adding more clusters.

This point is considered the optimal number of clusters as it provides a balance between minimizing WCSS and avoiding overfitting.

Silhouette Score:

The silhouette score is a measure of how similar a data point is to its assigned cluster compared to other clusters. It ranges from -1 to 1, with higher values indicating better clustering.

- **Score close to 1:** Indicates that the data point is well matched to its own cluster and poorly matched to neighboring clusters.
- **Score close to 0:** Indicates that the data point is on or very close to the decision boundary between two neighboring clusters.
- **Score close to -1:** Indicates that the data point might have been assigned to the wrong cluster.

The silhouette score can be used to evaluate the performance of various clustering algorithms and compare different choices of K. 3. Impact of Scaling and Outliers

Impact of Scaling:

Many clustering algorithms, including K-means, are sensitive to feature scaling. If features have different scales, features with larger values will dominate the distance calculations, leading to biased results.

- **Solution:** Feature scaling (e.g., standardization or normalization) ensures that all features contribute equally to the distance calculations. This often leads to improved clustering performance and more meaningful interpretations.

Impact of Outliers:

Outliers can significantly impact clustering results, particularly in K-means, as they can disproportionately influence centroid calculations.

- **Effect:** Outliers can pull cluster centroids towards themselves, leading to distorted clusters and potentially inaccurate assignments for other data points.
- **Solutions:**
 - Outlier detection and removal before clustering.
 - Using robust clustering techniques less sensitive to outliers (e.g., DBSCAN).

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
import matplotlib.cm as cm
!pip install ydata-profiling
from ydata_profiling import ProfileReport

import seaborn as sns
import math
from sklearn.cluster import KMeans
from sklearn.preprocessing import MinMaxScaler
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN
from sklearn.metrics import AgglomerativeClustering
import warnings
warnings.filterwarnings('ignore') # reading the data
df = pd.read_csv(f'{path}/Mall_Customers.csv')
df.head(3)
```



Requirement already satisfied: ydata-profiling in /usr/local/lib/python3.11/dist-pack
 Requirement already satisfied: scipy<1.16,>=1.4.1 in /usr/local/lib/python3.11/dist-p
 Requirement already satisfied: pandas!=1.4.0,<3.0,>1.1 in /usr/local/lib/python3.11/d
 Requirement already satisfied: matplotlib<=3.10,>=3.5 in /usr/local/lib/python3.11/di
 Requirement already satisfied: pydantic>=2 in /usr/local/lib/python3.11/dist-packages
 Requirement already satisfied: PyYAML<6.1,>=5.0.0 in /usr/local/lib/python3.11/dist-p
 Requirement already satisfied: jinja2<3.2,>=2.11.1 in /usr/local/lib/python3.11/dist-
 Requirement already satisfied: visions<0.8.2,>=0.7.5 in /usr/local/lib/python3.11/dis
 Requirement already satisfied: numpy<2.2,>=1.16.0 in /usr/local/lib/python3.11/dist-
 p
 Requirement already satisfied: htmlmin==0.1.12 in /usr/local/lib/python3.11/dist-pack
 Requirement already satisfied: phik<0.13,>=0.11.1 in /usr/local/lib/python3.11/dist-p
 Requirement already satisfied: requests<3,>=2.24.0 in /usr/local/lib/python3.11/dist-
 Requirement already satisfied: tqdm<5,>=4.48.2 in /usr/local/lib/python3.11/dist-pack
 Requirement already satisfied: seaborn<0.14,>=0.10.1 in /usr/local/lib/python3.11/dis
 Requirement already satisfied: multimethod<2,>=1.4 in /usr/local/lib/python3.11/dist-
 Requirement already satisfied: statsmodels<1,>=0.13.2 in /usr/local/lib/python3.11/di
 Requirement already satisfied: typeguard<5,>=3 in /usr/local/lib/python3.11/dist-pack
 Requirement already satisfied: imagehash==4.3.1 in /usr/local/lib/python3.11/dist-pac
 Requirement already satisfied: wordcloud>=1.9.3 in /usr/local/lib/python3.11/dist-pac
 Requirement already satisfied: dacite>=1.8 in /usr/local/lib/python3.11/dist-packages
 Requirement already satisfied: numba<=0.61,>=0.56.0 in
 /usr/local/lib/python3.11/dist
 Requirement already satisfied: PyWavelets in /usr/local/lib/python3.11/dist-packages
 Requirement already satisfied: pillow in /usr/local/lib/python3.11/dist-packages (fro
 Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.11/dist-
 pack
 Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-pac
 Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-package
 Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-pa
 Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-pa
 Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-pack
 Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-pac
 Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.11/dist
 Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.1
 Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-package
 Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packa
 Requirement already satisfied: joblib>=0.14.1 in /usr/local/lib/python3.11/dist-packa
 Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.11/di
 Requirement already satisfied: pydantic-core==2.27.2 in /usr/local/lib/python3.11/dis
 Requirement already satisfied: typing-extensions>=4.12.2 in /usr/local/lib/python3.11

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/
 Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-package
 Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-p
 Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-p
 Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.11/dist-package
 Requirement already satisfied: attrs>=19.3.0 in /usr/local/lib/python3.11/dist-packag
 Requirement already satisfied: networkx>=2.4 in /usr/local/lib/python3.11/dist-
 packag
 Requirement already satisfied: puremagic in /usr/local/lib/python3.11/dist-packages (
 Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (f

CustomerID Gender Age Annual Income (k\$) Spending Score (1-100)

0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female		20	16 6

```
# inspecting the data df.info()
df.isnull().sum() df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 200 entries, 0 to 199

Data columns (total 5 columns):

#	Column	Non-Null Count	Dtype
---	--------	----------------	-------

--- -----

0	CustomerID	200 non-null	int64
---	------------	--------------	-------

1	Gender	200 non-null	object
---	--------	--------------	--------

2	Age	200 non-null	int64
---	-----	--------------	-------

3	Annual Income (k\$)	200 non-null	int64	4	Spending Score (1-100)	200 non-null
---	---------------------	--------------	-------	---	------------------------	--------------

int64 dtypes: int64(4), object(1) memory usage: 7.9+ KB

CustomerID Age Annual Income (k\$) Spending Score (1-100)

count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	38.850000	60.560000	50.200000
std	57.879185	13.969007	26.264721	25.823522
min	1.000000	18.000000	15.000000	1.000000
25%	50.750000	28.750000	41.500000	34.750000
50%	100.500000	36.000000	61.500000	50.000000



75%	150.250000	49.000000	78.000000	73.000000
max	200.000000	70.000000	137.000000	99.000000

```
gender_counts = df['Gender'].value_counts(normalize=True) * 100
```

```
# Display print("Gender Split
(%):\n")
```

```
print(gender_counts)
```

```
profile = ProfileReport(df, title="Profiling Report") profile
```

Gender Split (%):

Gender

Female 56.0

Male 44.0

Name: proportion, dtype: float64

Summarize dataset: 100%

30/30 [00:02<00:00, 10.78it/s, Completed]

100%|██████████| 5/5 [00:00<00:00, 76.71it/s]

Generate report structure: 100%

1/1 [00:01<00:00, 1.57s/it]

Profiling Report

Overview

Brought to you by [YData](#)

Overview

[Alerts](#) 4[Reproduction](#)

Dataset statistics

Number of variables	5
Number of observations	200
Missing cells	0
Missing cells (%)	0.0%
Duplicate rows	0
Duplicate rows (%)	0.0%
Total size in memory	7.9 KiB
Average record size in memory	40.7 B

Variable types

Numeric	4
Categorical	1

Variables

```
plt.figure(figsize=(15, 5))
```

```
for i,
```

```
100)]]):
```

```
    plt.subplot(1, 3, i+1)    sns.histplot(data=df, x=col, bins=15,  
kde=True, color='skyblue')    plt.xlabel(col)
```

```
    plt.suptitle('Distribution of Age, Annual Income & Spending Score', fontsize=17, font  
plt.subplots_adjust(wspace=0.2)
```

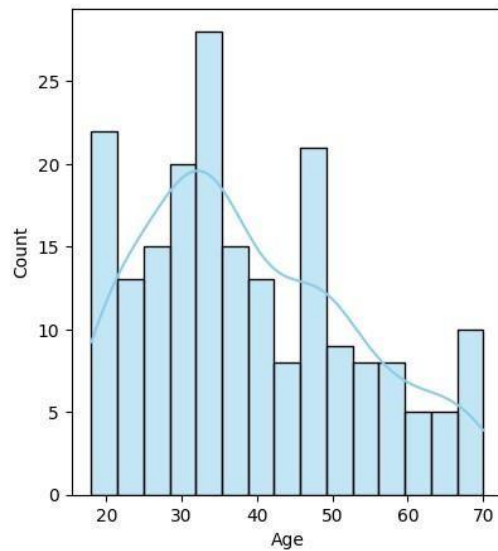
```
col in enumerate(['Age', 'Annual  
Income (k$)', 'Spending Score (1-
```



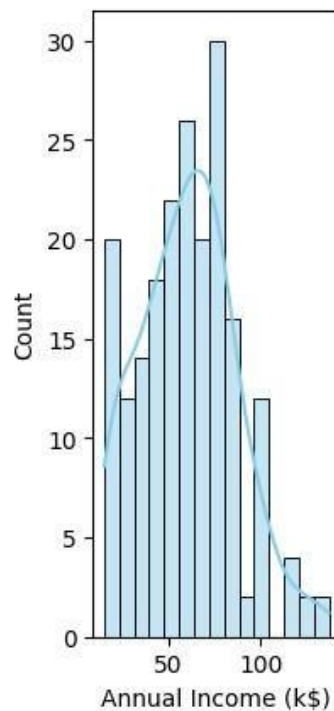
Display

`plt.show()`

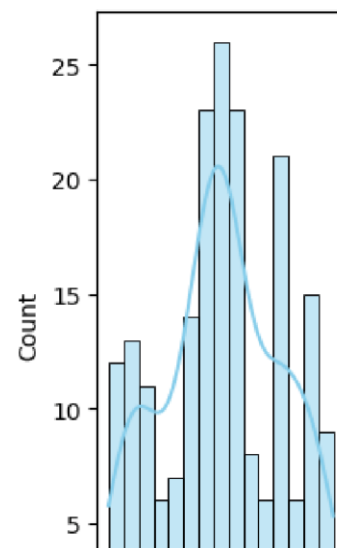
Distribution of Age, Annual Income & Spending Score



Distribution of Age, Annual Income & Spending Score



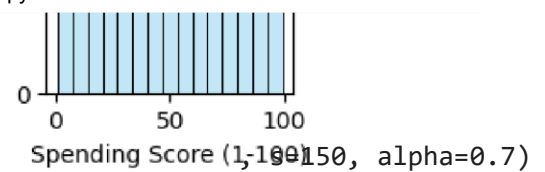
Distribution of Age, Annual Income & Spending Score



```
sns.set_style('whitegrid')  
plt.figure(figsize=(15, 6))
```



```
# Create Scatterplot
plt.figure(figsize=(15, 6))
```

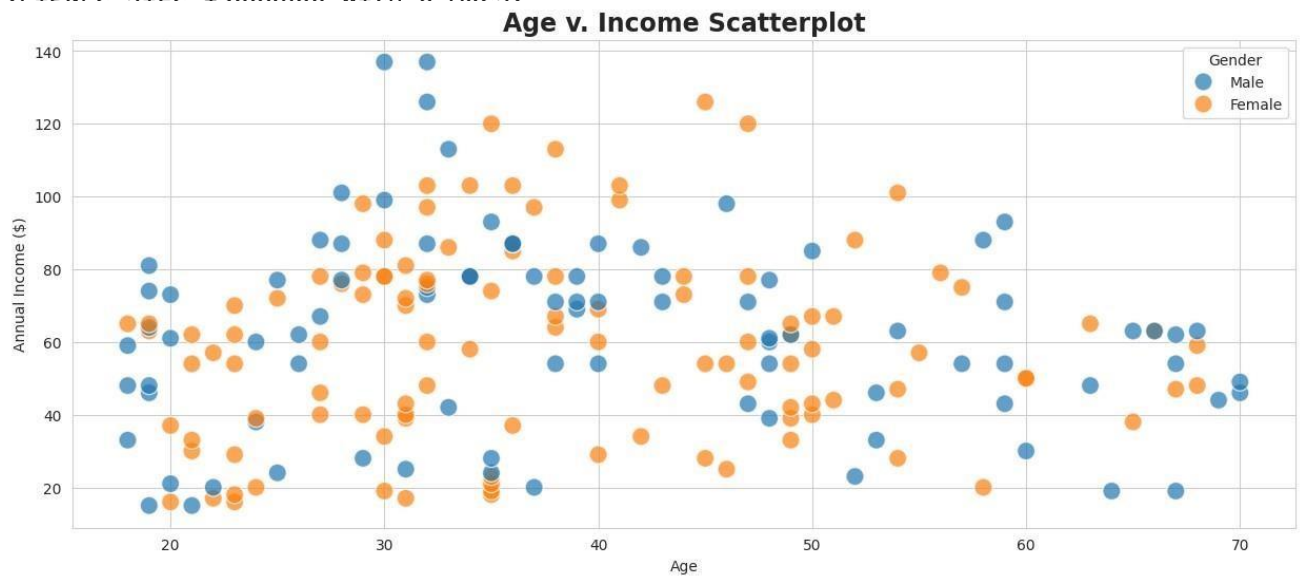


```
sns.scatterplot( x= 'Age', y= 'Annual Income (k$)', data=df , hue='Gender'
```

```
plt.xlabel('Age') plt.ylabel('Annual Income ($)') plt.title('Age v. Income Scatterplot',
fontsize=17, fontweight = 'bold') # Display plt.show
```



<Figure size 1500x600 with 0 Axes>



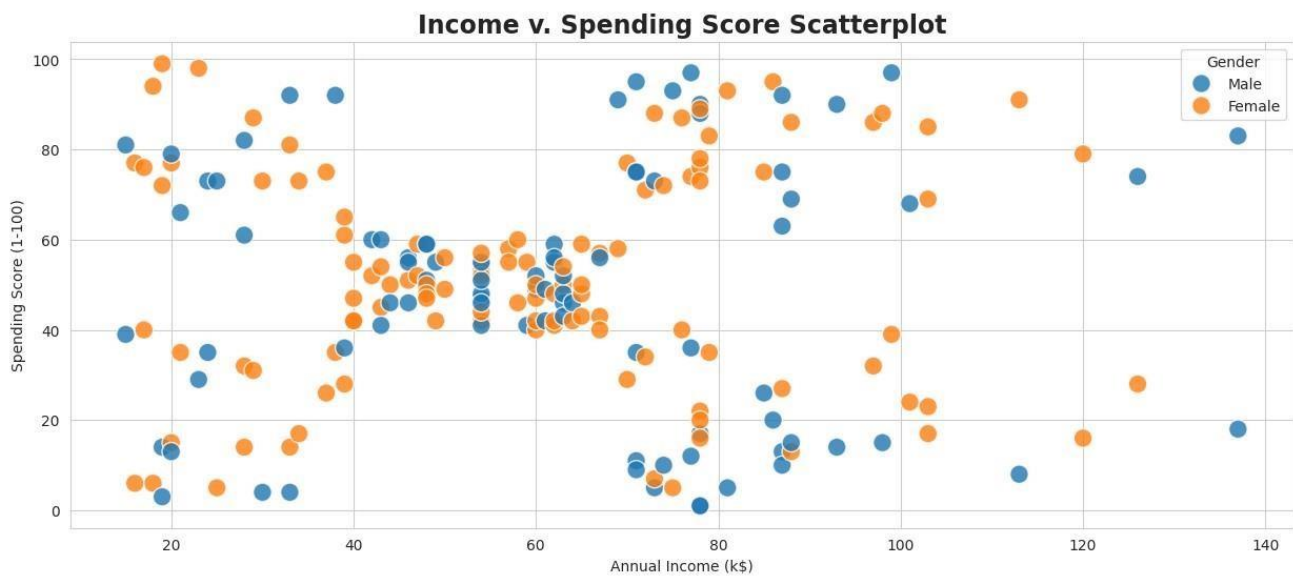
()

```
# Set style sns.set_style('whitegrid')
plt.figure(figsize=(15, 6))
```

```
# Create scatter plot of Annual Income and Age, colored by Gender sns.scatterplot(data=df,
x='Annual Income (k$)', y='Spending Score (1-100)', hue='Gender'
```

```
# Add title plt.title('Income v. Spending Score Scatterplot', fontsize=17, fontweight='bold')
```

```
# Display plt.show()
```



```
# Set style sns.set_style('whitegrid')  
plt.figure(figsize=(15, 8))
```

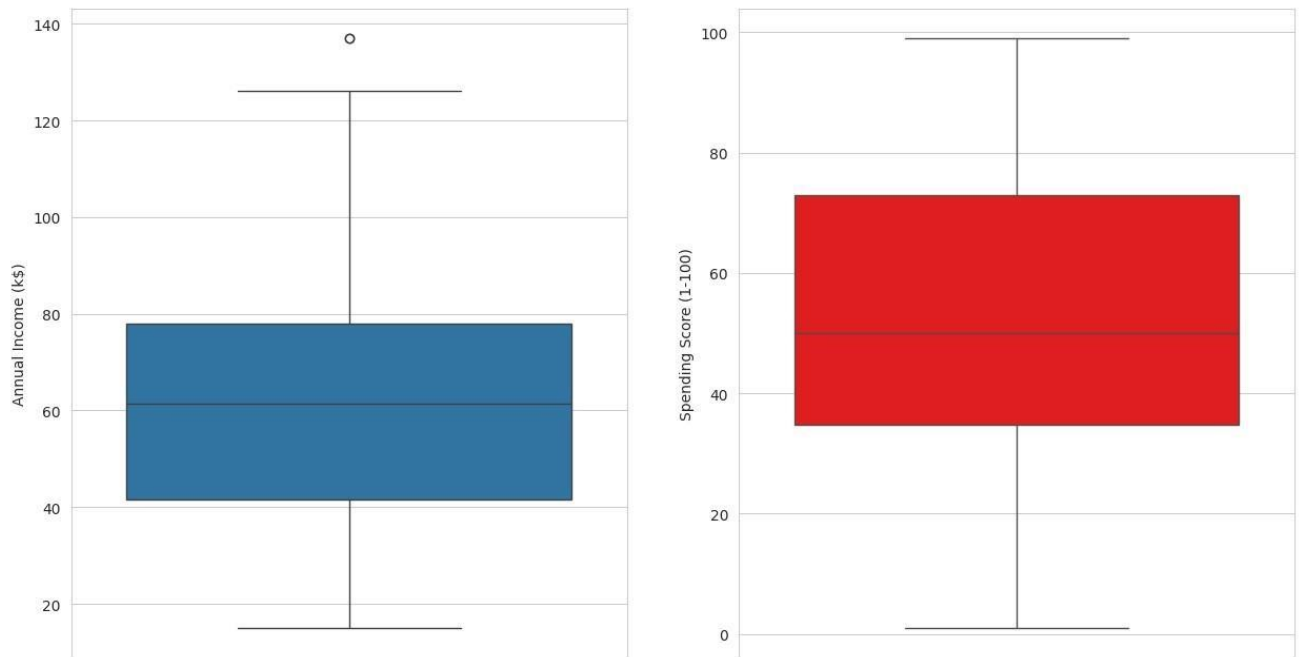
```
# Create boxplots plt.subplot(1, 2, 1)  
sns.boxplot(y=df["Annual Income (k$)"]) plt.subplot(1, 2,  
2) sns.boxplot(y=df["Spending Score (1-100)"],  
color='red')
```

```
# Add title plt.suptitle('Income v. Spending Score Distribution', fontsize=17,  
fontweight='bold', x=0)
```

```
# Display plt.show()
```



Income v. Spending Score Distribution

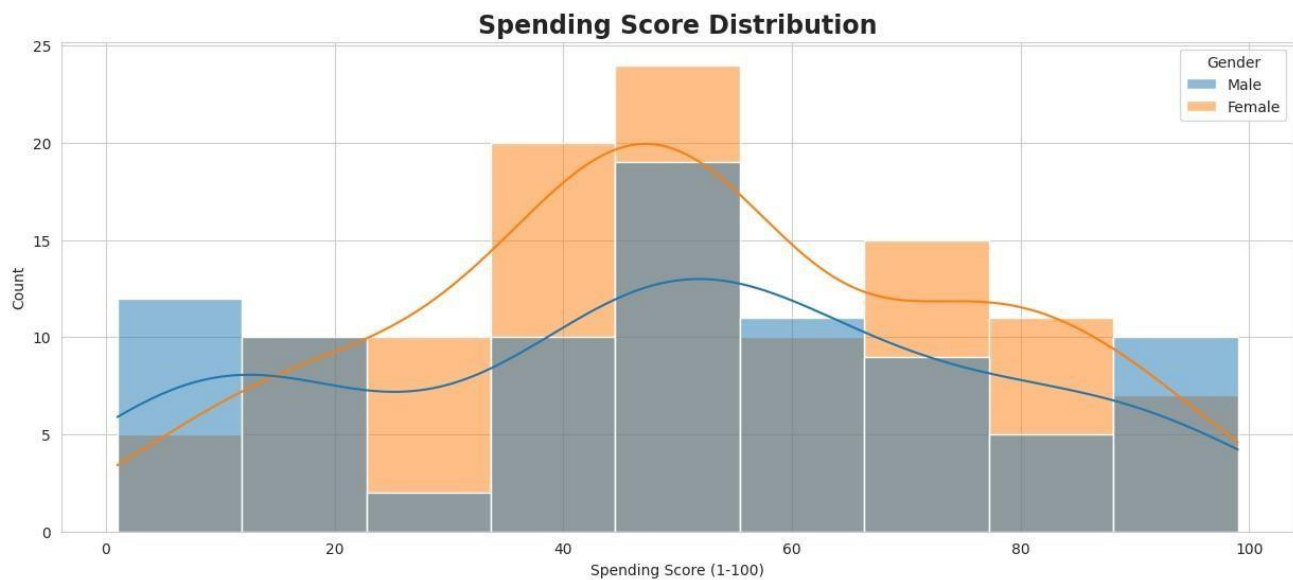


```
# Set style sns.set_style('whitegrid')  
plt.figure(figsize=(15, 6))
```

```
# Create Histogram sns.histplot(data=df, x='Spending Score (1-100)', hue='Gender',  
kde=True)
```

```
# Add title plt.title('Spending Score Distribution', fontsize=17, fontweight='bold')
```

```
# Display plt.show()
```



Normalise data

```
# Select numerical columns to normalize num_cols = ['Age', 'Annual
Income (k$)', 'Spending Score (1-100)']
```

```
# Initialise scaler scaler =
MinMaxScaler()
```

```
# Normalise df[num_cols] =
scaler.fit_transform(df[num_cols]) df.head()
```



	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	Male	0.019231	0.000000	0.387755
1	2	Male	0.057692	0.000000	0.816327
2	3	Female	0.038462	0.008197	0.051020
3	4	Female	0.096154	0.008197	0.775510



4

5 Female 0.250000 0.016393 0.397959

```
# Create k-means object - 4 clusters
```

```
kmeans = KMeans(n_clusters=4)
```

```
# Fit k-means object to the data kmeans.fit(df[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']])
```

```
# Add cluster labels to original df df['Cluster'] = kmeans.labels_
```

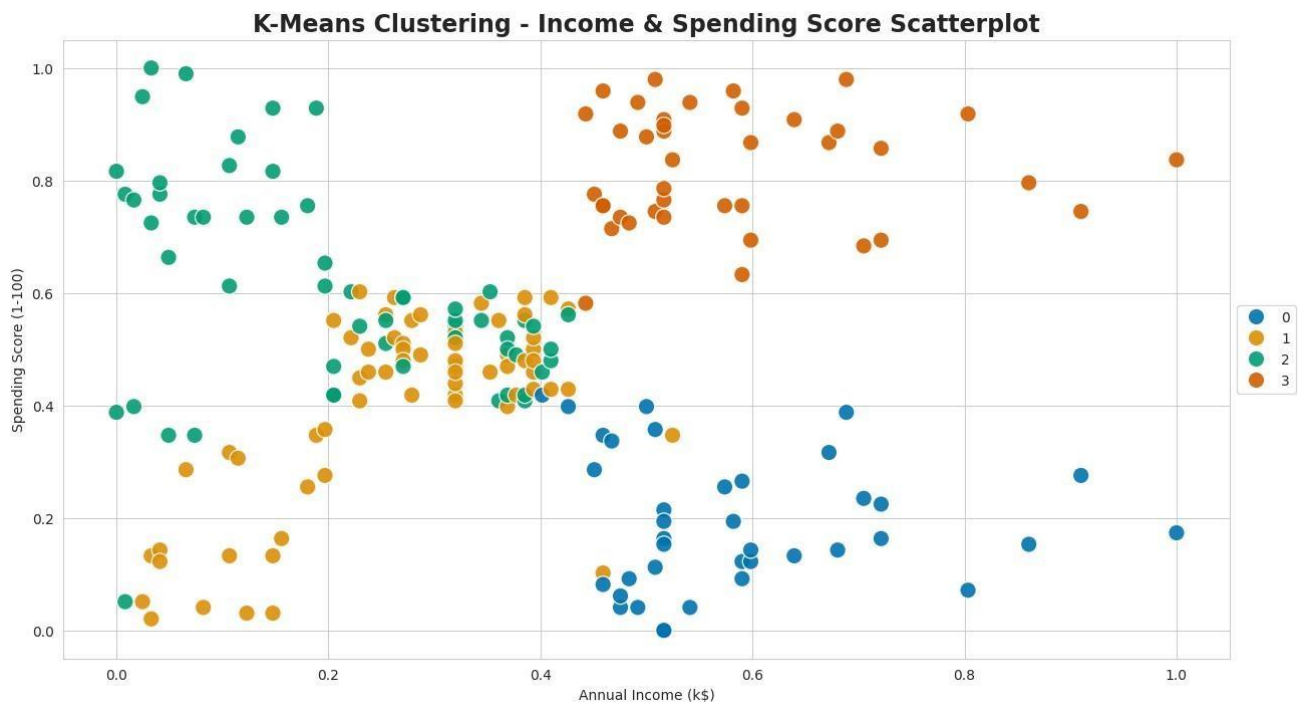
```
# Set style & figure size sns.set_style('whitegrid') plt.figure(figsize=(15, 8))
```

```
# Plot clusters sns.scatterplot(x='Annual Income (k$)', y='Spending Score (1-100)', data=df, hue='Cluster')
```

```
# Set title, axis labels & legend plt.title('K-Means Clustering - Income & Spending Score Scatterplot', fontsize=17, fontweight='bold') plt.xlabel('Annual Income (k$)') plt.ylabel('Spending Score (1-100)') plt.legend(loc='center left', bbox_to_anchor=(1, 0.5))
```

```
# Display
```

<matplotlib.legend.Legend at 0x78e08466f610>



```
centroids = kmeans.cluster_centers_
print(centroids)
```

```
[[0.41093117 0.58606557 0.1895811 ]
 [0.69201183 0.26809584 0.39764521]
 [0.14304993 0.20491803 0.60508414]
 [0.28605769 0.58278689 0.82168367]]
```

```
# Calculate silhouette score silhouette_avg = silhouette_score(df[['Age', 'Annual Income
(k$)', 'Spending Score (1-100)
print("The average silhouette_score is :", silhouette_avg)
```

↔ ↔ The average silhouette_score is : 0.39231920205572207

```
model = AgglomerativeClustering(n_clusters=4) model.fit(df[['Age',  
  
'Annual Income (k$)', 'Spending Score (1-100)']])  
df['Hierarchical_Cluster'] = model.labels_  
  
sns.set_style('whitegrid') plt.figure(figsize=(15, 8)) sns.scatterplot(x='Annual Income  
(k$)', y='Spending Score (1-100)', data=df, hue='Hierarc  
  
plt.title('Hierarchical Clustering - Income & Spending Score Scatterplot', fontsize=17, f  
plt.xlabel('Annual Income (k$)') plt.ylabel('Spending Score (1-100)') plt.legend(loc='center  
left', bbox_to_anchor=(1, 0.5))  
  
plt.show() print() silhouette_avg = silhouette_score(df[['Age', 'Annual Income (k$)',  
'Spending Score (1-100  
  
print("The average silhouette_score is :", silhouette_avg)  
print()  
  
sns.set_style('whitegrid') fig = plt.figure(figsize=(10, 8)) ax = fig.add_subplot(111,  
projection='3d') ax.scatter(df['Age'], df['Annual Income (k$)'], df['Spending Score (1-  
100)'], c=df['Hiera  
  
ax.set_title('3D Hierarchical Clustering', fontsize=17, fontweight='bold')  
ax.set_xlabel('Age') ax.set_ylabel('Annual Income (k$)') ax.set_zlabel('Spending  
Score (1-100)') plt.show()
```




The average silhouette_score is : 0.3652325990107206

